Tema 3: JSON

Índice

1. JSON 2
2. Elementos que componen un documento JSON 2
   2.1. Datos 2
   2.2. Tipos de valores 2
   2.3. Objetos JSON 3
   2.4. Arrays JSON 3
   2.5. Resumen 4
   2.6. JSON y XML 4
   2.7. Usos de JSON 4
   2.8. Herramientas 5
3. JSON y Java 5
   3.1. Configuración de proyecto y dependencias 5
   3.2. Fundamentos del Data Binding 6
   3.3. Modelado de DTO 6
   3.4. Anotaciones de datos 8
   3.5. Clase Central: ObjectMapper 8
   3.6. Lectura de un JSON a objetos (Deserialización) 9
   3.7. Escritura de objetos en un JSON (Serialización) 10
   3.8. Lectura, edición y escritura en un JSON 10
   3.9. Ejemplo completo 10
   3.10. Inspección dinámica sin DTO: Jackson Tree Model. 12
4. APIS de servicios web 15
   4.1. Frankfurter 15
   4.2. Open Weather Map 15
   4.3. Ticketmaster.com 16
   4.4. Last.fm 17
   4.5. Wikipedia 17
   4.6. IMDB 18

1. JSON

JSON (JavaScript Object Notation) es una alternativa, más sencilla de usar, a XML para el intercambio de datos.

Nació como un subconjunto de la notación de objetos de JavaScript. Debido a esto un programa JavaScript puede utilizar las funciones estándar, de JavaScript, para convertir datos JSON en objetos nativos de JavaScript, sin tener que utilizar un programa de análisis (como hace XML).

Las similitudes entre JSON y XML son:

- Tanto JSON y XML son texto plano, por lo tanto, legibles para los humanos.
- Tienen una sintaxis muy simple.
- Son jerárquicos (valores dentro de los valores).
- Independientes del lenguaje de programación.

Y sus diferencias son:

- JSON no utiliza etiqueta final.
- JSON es más corto.
- JSON es más rápido de leer y escribir.
- JSON puede utilizar matrices.
- JSON no tiene un esquema ampliamente aceptado para la definición y validación de la estructura de datos JSON.

2. Elementos que componen un documento JSON

2.1. Datos

Los datos se escriben como pares clave:valor.

Un par clave:valor consiste en un nombre de una clave, escrita entre comillas, seguido de dos puntos y un valor.

```
"Nombre": "Juan"
```

Los pares clave:valor se separan entre ellos por comas.

2.2. Tipos de valores

Los tipos de valores que podemos encontrar en JSON son los siguientes:

- Un número.
- Una cadena de cero o más caracteres unicode entre comillas.
- Booleano (verdadero o falso).
- Un array Json (entre corchetes).
- Un objeto Json (entre llaves).

- Null.

JSON representa solamente datos.No defineel concepto de variables, asignaciones, igualdades o funciones.

2.3. Objetos JSON

Los objetos JSON:

- Objetos se escriben entre llaves por lo que empiezan con { y terminan con }.
- Son un **conjunto desordenado** de pares clave:valor.
- Los pares clave:valor están separados, entre ellos, por comas.
- Las claves, parte izquierda del par, deben ser cadenas y deben ser diferentes entre sí.

Ejemplo:

```
{
    "nombre": "Manuela",
    "apellido": "Freire"
}
```

2.4. Arrays JSON

Las matrices JSON:

- Están encerradas entre corchetes por lo que una matriz comienza con [y termina con].
- Son una **lista ordenada** de valores.
- Cada valor puede ser de cualquier tipo, incluyendo objetos o arrays.
- Los valores están separados por comas.

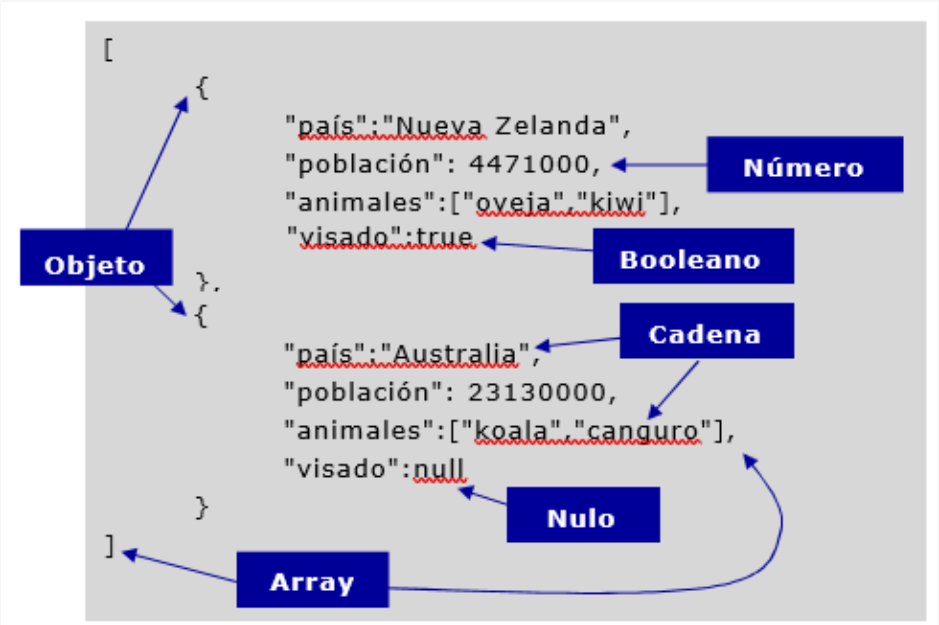
Ejemplo:

```
[
  {
    "nombre": "Manuela",
    "apellido": "Freire"
  },
  {
    "nombre": "Lorenzo",
    "apellido": "Veiga"
  },
  {
    "nombre": "Concepción",
    "apellido": "Senra"
  }
]
```

El ejemplo es un array que contiene tres valores que son tres objetos JSON. Cada objeto es un registro de una persona (con un nombre y un apellido).

2.5. Resumen

En el siguiente documento JSON podemos encontrar todos los elementos definidos anteriormente:



2.6. JSON y XML

En este ejemplo JSON se define un objeto empleados, que contiene un array con tres elementos. Cada uno de esos elementos es un objeto que representa un empleado:

```
{ "empleados": [
        {"nombre": "Manuela", "apellido": "Freire"},
        {"nombre": "Lorenzo", "apellido": "Veiga"},
        {"nombre": "Concepción", "apellido": "Senra"}
        {"nombre": "Manuel", "apellido": "Graiño"}
    ]
}
```

Veamos su equivalente XML:

```
<empleados>
      <empleado >
          <nombre>Manuela</nombre><apellido>Freire</apellido>
      </empleado>
      <empleado>
          <nombre>Lorenzo</nombre><apellido>Veiga</apellido>
      </empleado>
      <empleado >
          <nombre>Concepción</nombre><apellido>Senra</apellido>
      </empleado >
      <empleado>
          <nombre>Manuel</nombre><apellido>Graiño</apellido>
      </empleado>
</empleados>
```

2.7. Usos de JSON

JSON se usa a menudo como un formato común enviar información entre aplicaciones que se comunican entre sí a través de Internet. JSON es independiente de los lenguajes de programación usados para crear estas aplicaciones

así como de los entornos donde se ejecuten. Por lo que JSON es adecuado para este escenario, ya que es un estándar abierto, es fácil de leer y escribir, y es más compacto que otras representaciones.

Los servicios web RESTful utilizan ampliamente JSON como el formato de los datos dentro de las solicitudes y respuestas.

2.8. Herramientas

Para visualizar y comprobar la validez de un documento JSON existen varias herramientas online, entre las que podemos encontrar:

- http://jsonformatter.curiousconcept.com/
- http://jsonviewer.stack.hu
- http://jsonlint.com/

Eclipse, NetBeans, Visual Studio Code y Firefox permiten visualizar, de forma nativa, un documento JSON.

Notepad++ permite ver y formatear como un árbol un documento JSON. Deberemos instalar el plugin JSTool mediante la herramienta plugin manager que se encuentra en el Menú Plugins.

Una vez instalado podremos usar la opción JSON Viewer situado en el menú Plugins para formatear o ver el árbol de un texto seleccionado.

3. JSON y Java

Desde Java, podemos trabajar con archivos y documentos JSON mediante distintos modelos. Entre ellos:

- Modelo de objetos. Ofrece los tipos `JsonObject` y `JsonArray` para trabajar con los objetos y matrices JSON.
- Modelo de streaming, con un analizador basado en eventos.
- Modelo Data Binding. Asociar objetos JSON a clases nativas Java.

Tanto el modelo de objetos como el de streaming funcionan de forma similar a como se trabaja con XML en Java. Sin embargo, en el desarrollo profesional de aplicaciones que trabajan con JSON el modelo de Data Binding es el estándar de facto, a través de la librería **Jackson**[1]**.**

La ventaja es que **Jackson** permite trabajar con los tres modelos anteriores, aunque destaca sobretodo por su Data Binding.

En el desarrollo profesional de software Java, el estándar de facto para el trabajo con JSON es la suite de librerías **FasterXML Jackson.**

3.1. Configuración de proyecto y dependencias

[1] https://github.com/FasterXML/jackson

Para usar **Jackson** en cualquier proyecto Java, la forma recomendable es mediante un gestor de dependencias como **Maven**. Se debe incluir la siguiente dependencia.

```
<dependencies>
    <dependency>
        <groupId>com.fasterxml.jackson.core</groupId>
        <artifactId>jackson-databind</artifactId>
        <version>2.17.0</version>
    </dependency>
</dependencies>
```

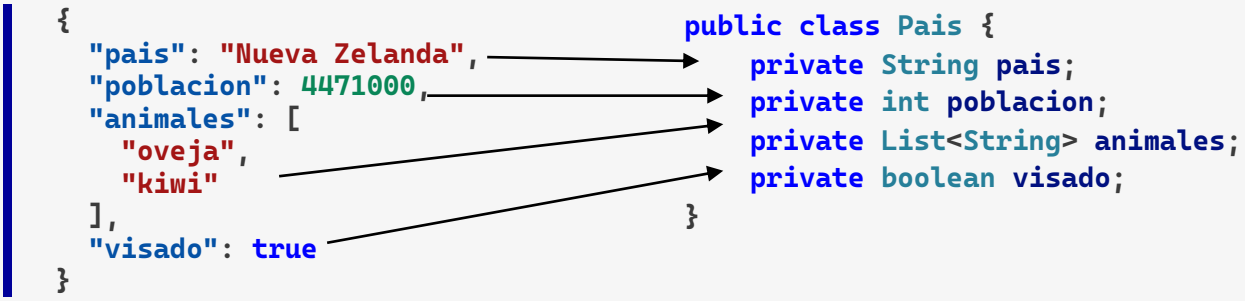
3.2. Fundamentos del Data Binding

El **Data Binding** (enlace o vinculación de datos) es el mecanismo por el cual una librería (en este caso Jackson) convierte de forma bidireccional texto estructurado en JSON a instancias de clases en memoria, y viceversa, mediante reflexión e introspección.

Es decir, traduce el JSON a una instancia de una clase JSON asignando sus campos que por defecto se espera que tengan el mismo nombre.

A estas clases de Java que usamos para representar un JSON se le suele llamar **Data Transfer Object (DTO)**, porque típicamente sólo sirven para almacenar los valores leídos de un JSON hacia nuestra aplicación, sin lógica de negocia compleja.

El siguiente gráfico muestra un ejemplo



En el caso de que un objeto JSON tuviese anidado otro objeto JSON, en la clase de Java sería tan simple como tener un campo como instancia de otra clase que modele el segundo objeto JSON.

3.3. Modelado de DTO

Para modelar un JSON como objetos DTO, es decir, como clases Java, basta con definir una jerarquía de clases Java que represente ese objeto JSON, utilizando los tipos de datos asimilables a cada una de las propiedades.

Por defecto, Jackson vincula cada propiedad JSON por su clave (nombre) suponiendo que coincide con el nombre del campo en la clase JAVA.

Así, en el ejemplo del punto 3.2:

- Propiedad JSON "país", que es un string → `String país` de la clase Java `Pais`.
- Propiedad JSON "población, entero → `Int poblacion` de la clase Java `Pais`.

- Propiedad JSON "animales", colección de strings → `List<String> animales` de la clase Java `Pais`. También valdría un `String[]` o `ArrayList<String>.`
- PropiedadJSON "visado", que es un boolean → `boolean visado` de la clase Java `Pais`.

Para modelar estas clases Java existen dos opciones:

Clases POJO

Las clases **Plain Old Java Object (POJO)** son las clases tradicionales de Java. Consiste en definir clases Java con las siguientes características:

- Definir campos con los nombres que coinciden con las propiedades JSON
- Definir getters y setters para cada una de las propiedades.
- Definir al menos un **constructor sin parámetros**. Jackson primero declara la instancia y posteriormente usa los getters y setters para *mappear* las propiedades.

```
public class PaisPOJO {
    private String pais;
    private int poblacion;
    private List<String> animales;
    private boolean visado;

    public PaisPOJO() {
    }

    public String getPais() {
        return pais;
    }
    public void setPais(String pais) {
        this.pais = pais;
    }
    public int getPoblacion() {
        return poblacion;
    }
    public void setPoblacion(int poblacion) {
        this.poblacion = poblacion;
    }
    public List<String> getAnimales() {
        return animales;
    }
    public void setAnimales(List<String> animales) {
        this.animales = animales;
    }
    public boolean getVisado() {
        return visado;
    }
    public void setVisado(boolean visado) {
        this.visado = visado;
    }
}
```

Records Java (Java +16)

Los Records (introducidos en Java 16) son un tipo especial de clase diseñado para **portar datos inmutables**, lo que los hace perfectos para los DTO. Su objetivo es eliminar todo este código repetitivo de los POJO tradicionales. Se caracterizan por:

- Se definen de forma compacta.
- Son inmutables, pues una vez creado el objeto no se pueden cambiar los valores (no existen setters).

- El compilador automáticamente genera:
  - Declara cada parámetro como privado y final.
  - El constructor para todos los parámetros.
  - Métodos de lectura, sin el prefijo get.

```
public record PaisDto(
        String pais,
        int poblacion,
        List<String> animales,
        boolean visado) {
}
```

3.4. Anotaciones de datos

Para personalizar los nombres de las propiedades, **Jackson** proporciona un conjunto de anotaciones que permiten desvincular el nombre que le damos al campo de nuestra clase Java del JSON, así como otros usos.

Anotación	Propósito
@JsonProperty("nombre")	Mapea una clave JSON con un nombre diferente al del atributo Java. Imprescindible para convertir formatos snake_case de APIs (ej. "created_at") a camelCase (createdAt), o para nombres con caracteres conflictivos (ej. "año").
@JsonIgnoreProperties(ignoreUnknown = true)	Evita que Jackson lance la excepción UnrecognizedPropertyException cuando el JSON contiene campos adicionales que no nos interesa modelar en el DTO. Se usa sobre la clase
@JsonInclude(JsonInclude. Include.NON_NULL)	Al serializar a JSON, excluye del resultado aquellos campos que tengan valor null, produciendo JSONs más limpios y compactos.
@JsonFormat(pattern = "...")	Define el formato exacto de serialización de fechas y horas (por ejemplo: yyyy-MM-dd HH:mm:ss).

3.5. Clase Central: ObjectMapper

La clase `com.fasterxml.jackson.databind.ObjectMapper` es la clase principal de **Jackson** que permite tanto mapear un JSON a clases Java (deserializar) como escribir el contenido de clases Java a JSON. Se debe tener en cuenta:

- La inicialización es computacionalmente costosa, pero una vez creada es totalmente seguro reutilizarla. La práctica habitual es reutilizarla siempre que se pueda como una instancia estática o un Singleton.

- Permite configurar parámetros globales sobre cómo se maneja la serialización y deserialización, por ejemplo ignorando siempre propiedades no mapeadas en lugar de tener que usar `@JsonIgnoreProperties` en cada clase DTO.

Una opción es crear una clase utility que administre una instancia ObjectMapper estática como la siguiente:

```
import com.fasterxml.jackson.databind.DeserializationFeature;
import com.fasterxml.jackson.databind.ObjectMapper;

public class JsonTools {

    private static ObjectMapper mapper;

    public static ObjectMapper getMapper() {
        if(mapper != null)
            return mapper;

        mapper = new ObjectMapper();

        // Configura para ignorar propiedades desconocidas de forma global.
        mapper.disable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES);

        // Otras configuraciones.
        return mapper;
    }
}
```

3.6. Lectura de un JSON a objetos (Deserialización)

Para leer un JSON y transformarlo a objetos, se usa el método `readValue` de `ObjectMapper`. Tiene varias sobrecargas, pero la mayoría comparten:

- Primer parámetro: el JSON, que puede venir de un `Stream`, `File` o incluso directamente el json en forma de `String`, entre otro
- Segundo parámetro: la clase a la que se pretende deserializar el JSON.

El JSON del apartado 2.5, por ejemplo, se podría deserializar a un array de la clase PaisDto. A continuación, se muestra el ejemplo de realizar el mapeo desde un fichero y desde una Web API

```
ObjectMapper mapper = JsonTools.getMapper();

// Desde un fichero
File file = new File("ruta_a_paises.json");
PaisDto[] paisesDesdeFichero = mapper.readValue(file, PaisDto[].class);

// Desde una web api.
URL rutaAJson = URI.create("https://api.example.com/paises").toURL();
try(InputStream inputStream = rutaAJson.openStream()) {
    PaisDto[] paisesDesdeWeb = mapper.readValue(inputStream, PaisDto[].class);
}
```

3.7. Escritura de objetos en un JSON (Serialización)

Para escribir una jerarquía de objetos a un JSON, se usa el método `writeValue` de `ObjectMapper`. Tiene varias sobrecargas, pero la mayoría comparten:

- Primer parámetro: a dónde escribir el JSON resultado, que puede ser un `Stream`, `File` o similares.
- Segundo parámetro: la instancia que se quiere serializar.
- Para convertir directamente a string, usamos `writeValueAsString`

El siguiente ejemplo muestra como convertir un objeto PaisDto a un JSON que se guarda en un archivo.

```
PaisDto pais = new PaisDto("España", 498015590, null, true);
mapper.writeValue(new File("ruta_json_españa.json"), pais);
```

```
{"país":"España","población":498015590,"animales":null,"visado":true}
```

Por defecto, el resultado es un JSON minimizado, aunque se puede guardar con identación usando las configuraciones de `ObjectMapper`.

```
// Habilita la identación al escribir JSON
mapper.enable(SerializationFeature.INDENT_OUTPUT);
```

3.8. Lectura, edición y escritura en un JSON

Si quisiésemos leer datos de un JSON, editar algunos de ellos y volver a guardar el resultado, tenemos varias alternativas.

- Usando **records** es un proceso laborioso. Como son inmutables, no podemos editar la propiedad deseada y guardar el resultado. La opción sería volver a construir todo el árbol de objetos cambiando sólo los datos necesarios.
- Usando clases **POJO** es muy sencillo, pues se deserializa, se usan los *setters* necesarios para editar los valores y se serializa de nuevo a un archivo.
- Accediendo directamente al nodo que necesitamos editar, sin usar mapeo de clases, como veremos en el siguiente punto.

Cuando diseñamos los DTO para realizar el mapeo de clases, resulta conveniente valorar el uso que les vamos a dar. Usar records es ideal para escenarios de solo lectura mientras que si planeamos editar, deberemos optar por clases **POJO**.

3.9. Ejemplo completo

En el siguiente ejemplo, se muestra como usar objetos DTO para escribir el siguiente Json.

Este es el JSON que queremos reproducir.

```
[
  {
    "titulo": "El atlas de las nubes",
    "año": 2012,
    "directores": "Lana Wachowski, Lilly Wachowski",
    "interpretes": [
      {
        "nombre": "Tom Hanks",
        "fechaNacimiento": {
          "año": 1956,
          "mes": 8
        }
      },
      {
        "nombre": "Halle Berry",
        "fechaNacimiento": {
          "año": 1966,
          "mes": 7
        }
      }
    ]
  },
  {
    "titulo": "La red social",
    "año": 2010,
    "directores": "David Fincher",
    "interpretes": [
      {
        "nombre": "Jesse Eisenberg",
        "fechaNacimiento": {
          "año": 1983,
          "mes": 9
        }
      },
      {
        "nombre": "Andrew Garfield",
        "fechaNacimiento": {
          "año": 1983,
          "mes": 7
        }
      }
    ]
  }
]
```

Establecemos los DTO que mapean el modelo JSON.

```
public record PeliculaDto(
    String titulo,
    @JsonProperty("año") int year,
    String directores,
    InterpreteDto[] interretes
) {
}

public record InterpreteDto(
    String nombre,
    FechaNacimientoDto fechaNacimiento
){
}

public record FechaNacimientoDto(
    @JsonProperty("año") int year,
    int mes
){
}
```

Y el siguiente código, permite rellenar un árbol de objetos DTO, escribirlo en un archivo `películas.json` y recuperarlo desde el archivo.

```
// Declarar interpretes
InterpreteDto tomHanks = new InterpreteDto(
        "Tom Hanks",
        new FechaNacimientoDto(1956, 8));
InterpreteDto halleBerry = new InterpreteDto(
        "Halle Berry",
        new FechaNacimientoDto(1966, 7));
InterpreteDto jessEisenberg = new InterpreteDto(
        "Jess Eisenberg",
        new FechaNacimientoDto(1983, 5));
InterpreteDto andrewGarfield = new InterpreteDto(
        "Andrew Garfield",
        new FechaNacimientoDto(1984, 8));

// Peliculas
PeliculaDto laRedSocial = new PeliculaDto(
        "La Red Social",
        2010,
        "David Fincher",
        new InterpreteDto[] { jessEisenberg, andrewGarfield });
PeliculaDto atlasNubes = new PeliculaDto(
        "El atlas de las nubes",
        2012,
        "Lana Wachowski, Lilly Wachowski",
        new InterpreteDto[] { tomHanks, halleBerry });


// Escribir en archivo.
String rutaPelicula = "src/main/resources/peliculas.json";
mapper.writeValue(
    new File(rutaPelicula),
    new PeliculaDto[] { laRedSocial, atlasNubes });


// Si queremos recuperar los datos:
PeliculaDto[] peliculas = mapper.readValue(
    new File(rutaPelicula),
    PeliculaDto[].class);
System.out.println("Peliculas recuperadas:");
for (PeliculaDto pelicula : peliculas) {
    System.out.println("Titulo: " + pelicula.titulo());
    System.out.println("Director: " + pelicula.directores());
    System.out.println();
}
```

3.10. Inspección dinámica sin DTO: Jackson Tree Model.

Existen situaciones donde crear clases DTO no resulta práctico:

- Cuando el JSON tiene una estructura muy cambiante o desconocida.
- Cuando sólo necesitamos un par de datos puntuales de un JSON con muchos niveles de anidamiento.
- Cuando queremos editar un único dato puntual de un JSON que cumple alguna de las características anteriores.

Para estos escenarios, **Jackson** ofrece el **Tree Model** (modelo de árbol en memoria) que funciona de forma similar al modelo DOM de XML. La ventaja es que Jackson lo simplifica mucho para que sea muy fácil de usar.

En este modelo, todo el árbol se organiza en tres tipos:

- `ObjectNode`. Representa un nodo que es un objeto, como se indicaba en el apartado 2.
- `ArrayNode`. Representa un nodo que es un array. Dentro contendrá `ObjectNode`.
- `JsonNode`. Clase base que se puede castear a alguna de las anteriores, aunque podría ser nodos simples en el caso de un ArrayNode que solo contenga primitivos.

Lectura y navegación por árbol JSON

Para cargar un JSON en un árbol, usamos un método similar a `readValue()`, el método `readTree()`.

Para consultar los campos existen dos métodos fundamentales:

- `nodo.get("clave").` Devuelve el `JsonNode` asociado o null si no existe. Encadenar llamadas `get()` puede provocar `NullPointerException`.
- `nodo.path("clave").` Es la forma recomendada, si la clave no existe, devuelve un objeto especial llamado MissingNode, lo que permite encadenar llamadas de forma segura sin riesgo.

Para extraer el valor final como tipo primitivo se emplean métodos como `.asText(), .asint(), .asBoolean()`…

Siguiendo el ejemplo de películas, podríamos acceder a los nombres de los intérpretes de la siguiente forma.

```
// Leer como Tree Node
JsonNode peliculasArbol = mapper.readTree(new File(rutaPelicula));

// Acceder a un elemento que existe
String nombreInterprete =
peliculasArbol.path(0).path("interpretes").path(1).path("nombre").asText();
System.out.println("El nombre del segundo intérprete de la primera pelicula es: " +
nombreInterprete);

// Si no existiera algun nombre intermedio, no rompe con excepción
String nombreInterpreteNoExiste =
peliculasArbol.path(4).path("interpretes").path(1).path("nombre").asText();
System.out.println("El nombre del segundo intérprete de la cuarta pelicula es: " +
nombreInterpreteNoExiste);


// En cambio esto si que lanza NullPointerException
try {
    nombreInterpreteNoExiste =
peliculasArbol.get(4).get("interpretes").get(1).get("nombre").asText();
} catch (NullPointerException e) {
    System.out.println("Excepcion debido a que no existe");
}
```

Escritura de nodo

Si necesitásemos alterar un dato existente podemos, debemos capturar el nodo y loconvertimos a su tipo específico mutable (ObjectNode para objetos, ArrayNode para arrays) mediante un cast.

- Si únicamente hay que cambiar un valor primitivo, podemos usar `nodo.put()`. Ejemplo: cambiar el nombre de `David Fincher` por `David Andrew Leo Fincher.` Como sabemos que está en la segunda posición del array, accedemos directamente a él por posición (Opción A). SI no lo supiéramos, deberíamos iterar sobre el array hasta encontrarlo por título (ver opción B). En cualquier caso, finalizamos guardándolo.

  También podemos usar `nodo.remove("propertyName")` para eliminar propiedades.

```
// Opcion A: Si sabemos que está en la posición 0, lo cogemos usando path(0);
ObjectNode nodoLaRedSocial = (ObjectNode)peliculasArbol.path(1);
nodoLaRedSocial.put("directores", "David Andrew Leo Fincher");

//Opcion B: Alternativa buscarlo por nombre y usar put
ArrayNode arbolComoNodo = (ArrayNode)peliculasArbol;
for (JsonNode pelicula : arbolComoNodo) {
    String nombre = pelicula.path("titulo").asText();
    if(nombre.toLowerCase() == "La Red Social"){
        ObjectNode peliculaNodo = (ObjectNode)pelicula;
        peliculaNodo.put("directores", "David Andrew Leo Fincher");
    }
}

// Guardar el árbol modificado en un archivo
mapper.writeValue(new File("src/main/resources/peliculas_modificado.json"),peliculasArbol);
```

- Si es necesario escribir un ObjectNode, lo más habitual es usar el objeto DTO para cargarlo. Una instancia se puede convertir directamente a un JsonNode usando mapper.asJsono En el ejemplo, añadimos sobre el ArrayNode principal, una nueva película.

```
// Añadimos una nueva película de David fincher.
// Por simplicidad, ignoramos los intérpretes
PeliculaDto clubDeLaLucha = new PeliculaDto(
    "El club de la lucha",
    1999,
    "David Fincher",
    new InterpreteDto[]{}
);

JsonNode clubComoNodo = mapper.valueToTree(clubDeLaLucha);

ArrayNode arrayPeliculas = (ArrayNode)peliculasArbol;
arrayPeliculas.add(clubComoNodo);
```

4. APIS de servicios web

Veremos ahora unas cuantas listas con servicios web que se pueden usar:

- Buscador de datos ofrecidos por el Gobiernos Español. https://datos.gob.es/es/catalogo?res_format_label=JSON
- Buscador de datos ofrecidos por la Xunta de Galicia. https://abertos.xunta.gal/busca-de-datos
- Servicios ofrecidos por MeteoGalicia. https://www.meteogalicia.gal/web/rss-georss-json https://www.meteogalicia.gal/web/RSS/rssIndex.action?request_locale=gl
- Listado de APIs públicas para usar en el desarrollo de software. https://github.com/toddmotto/public-apis

De las siguientes veremos una pequeña descripción de su utilización:

4.1. Frankfurter

Frankfurter es una API https://frankfurter.dev/es/ sencilla ideal para empezar a probar a consumir apis. Proporciona datos actuales e históricos de tipos de cambio de divisas publicados por los Bancos Centrales. No requiere registro ni clave API.

Entr las opciones encontramos:

- Obtener todos los tipos de cambio, desde una divisa a todas las demás:

  https://api.frankfurter.dev/v2/rates?base=EUR
- Obtener los tipos de cambio desde una divisa a otras en particular:

  https://api.frankfurter.dev/v2/rates?base=EUR&quotes=USD,GBP,JPY
- Obtener la evolución del cambio de una divisa a otras en una serie temporal, especificando resolución de una semana:
- https://api.frankfurter.dev/v2/rates?base=EUR&quotes=USD&from=2026-01-01&to=2026-10-18&group=week

4.2. Open Weather Map

Openweathermap nos ofrece una completa API http://openweathermap.org/api para poder obtener el pronóstico del tiempo en una o varias localidades.

Por defecto nos ofrece la información en formato JSON pero mediante el parámetro &mode=xml podemos obtener el XML equivalente.

Otro parámetro interesante es &units=metric que convierte todas las temperaturas a grados centígrados.

En todas nuestras consultas usaremos a API KEY generada para pruebas de la siguiente forma:

&APPID=a975f935caf274ab016f4308ffa23453

Entre las opciones que disponemos se encuentran:

- Listar el tiempo de una localidad por su nombre, se le puede añadir (es opcional) el código de país para determinar el país de la localidad:

  http://api.openweathermap.org/data/2.5/weather?q=vigo,es&lang=es&APPID=a975f935caf274ab016f4308ffa23453

- Listar el tiempo de una localidad por su id:

  http://api.openweathermap.org/data/2.5/weather?id=3105976&lang=es&APPID=a975f935caf274ab016f4308ffa23453

- Utilizando su longitud y latitud:

  http://api.openweathermap.org/data/2.5/weather?lat=42.232819&lon=-8.72264&APPID=a975f935caf274ab016f4308ffa23453

- N predicciones más próximas a una un punto:

  http://api.openweathermap.org/data/2.5/find?lat=42.232819&lon=-8.72264&cnt=20&APPID=a975f935caf274ab016f4308ffa23453

4.3. Ticketmaster.com

Ticketmaster.com proporciona un servicio web que ofrece información de eventos, locales, … Su API está disponible en:

https://developer.ticketmaster.com/products-and-docs/apis/discovery-api/v2/

Para realizar búsquedas deberemos disponer de una key válida. Para pruebas podemos usar la API KEY:

apikey=AMXR5Rf8zlr7oGucsebGKvDCLOQmGUGE

Ejemplos de búsquedas son:

- Eventos en España:

  https://app.ticketmaster.com/discovery/v2/events.json?countryCode=ES&apikey=AMXR5Rf8zlr7oGucsebGKvDCLOQmGUGE

- Eventos en España por palabra clave:

  https://app.ticketmaster.com/discovery/v2/events.json?countryCode=ES&keyword=celta&apikey=AMXR5Rf8zlr7oGucsebGKvDCLOQmGUGE

- Lista de eventos clasificándolo por tipo, en esta caso música, en España:

  https://app.ticketmaster.com/discovery/v2/events.json?classificationName=music&countryCode=ES&apikey=AMXR5Rf8zlr7oGucsebGKvDCLOQmGUGE

- Lista de 30 eventos deportivos en España:

  https://app.ticketmaster.com/discovery/v2/events.json?countryCode=ES&classificationName=sports&apikey=AMXR5Rf8zlr7oGucsebGKvDCLOQmGUGE&size=30

- Datos de un evento en particular:

  https://app.ticketmaster.com/discovery/v2/events/LvZ18QtH-D1JheB5_oEcj.json?apikey=AMXR5Rf8zlr7oGucsebGKvDCLOQmGUGE

- Datos de un artista, equipo, … en particular:

- https://app.ticketmaster.com/discovery/v2/attractions/K8vZ9179pHf.json?apikey=AMXR5Rf8zlr7oGucsebGKvDCLOQmGUGE
- Lista de locales, estadios, … de España:

  https://app.ticketmaster.com/discovery/v2/venues.json?countryCode=ES&apikey=AMXR5Rf8zlr7oGucsebGKvDCLOQmGUGE
- Lista de locales, estadios, …que tengan una palabra clave:

  https://app.ticketmaster.com/discovery/v2/venues.json?&keyword=vigo&apikey=AMXR5Rf8zlr7oGucsebGKvDCLOQmGUGE
- Datos de un local en concreto:

  https://app.ticketmaster.com/discovery/v2/venues/Z198xZ2qZ1a1?apikey=AMXR5Rf8zlr7oGucsebGKvDCLOQmGUGE

4.4. Last.fm

LastFM ofrece una API http://www.lastfm.es/api/rest que proporciona todo tipo de información sombre artistas, grupos musicales, álbumes, canciones e incluso geolocalización de actividades cercanas a una longitud y latitud.

Nos ofrece información tanto en formato XML como en formato JSON; por defecto los resultados están en formato XML, para recuperar los resultados en formato JSON hay que añadir el parámetro: &format=json.

Para realizar consultas en LastFM se requiere estar registrado y contar con una API KEY válida. Podemos usar la siguiente API KEY:

&api_key=7ba9be9fd8a0a0c23c81c9e368c7e120

Entre otras, nos ofrece las siguientes búsquedas:

- Buscar por álbum:

  http://ws.audioscrobbler.com/2.0/?method=**album.search&album=believe**&api_key=7ba9be9fd8a0a0c23c81c9e368c7e120&format=json
- Información de un álbum: http://ws.audioscrobbler.com/2.0/?method=album.getinfo&artist=Cher&album=Believe&api_key=7ba9be9fd8a0a0c23c81c9e368c7e120&format=json
- Información de un artista: http://ws.audioscrobbler.com/2.0/?method=artist.getinfo&artist=Cher&api_key=7ba9be9fd8a0a0c23c81c9e368c7e120&format=json
- Buscar por artista:

  http://ws.audioscrobbler.com/2.0/?**method=artist.search&artist=cher**&api_key=7ba9be9fd8a0a0c23c81c9e368c7e120&format=json

4.5. Wikipedia

En la siguiente página https://www.mediawiki.org/wiki/API:Main_page/es podemos ver la API de Wikipedia para realizar consultas contra ella. Podemos consultar los parametros necesarios para hacer búsquedas en http://commons.wikimedia.org/w/api.php.

- Búsqueda de tópicos http://commons.wikimedia.org/w/api.php?action=opensearch&search=vigo&limit=50
- Acceso a un tópico

  http://en.wikipedia.org/w/api.php?format=json&**action=query&titles=vigo**&prop=revisions&rvprop=content

4.6. IMDB

Mediante la web http://www.omdbapi.com se puede buscar en la base de datos de IMDB (www.imdb.com) usando los siguientes criterios de búsqueda:

Parámetro	Descripción
s	String - Búsqueda por título de la película
y	String - Filtra una búsqueda por año
i	String - Devuelve película por su id IMDB
t	String - Devuelve película por su título
r	JSOM / XML - Tipo de datos a devolver JSOM o XML
plot	Short / full - Argumento de la película.
tomatoes	True - Añade información de: http://www.rottentomatoes.com

Y usando la siguiente API KEY generada para pruebas:

&apikey=75fe1286

Ejemplos de búsquedas son:

- Búsqueda por películas que en él titulo contenga la cadena eleven: http://www.omdbapi.com/?**s=eleven**&apikey=75fe1286
- Búsqueda la película con titulo Ocean's Eleven en el año 2001: http://www.omdbapi.com/?t=Ocean%27s%20Eleven&y=2001&apikey=75fe 1286
- Búsqueda a película con titulo Ocean's Eleven del año 2001 y devuelve su argumento (plot) junto con los datos de rotten tomatoes: http://www.omdbapi.com/?**t=Ocean'sEleven&y=2001&plot=full&tomatoes=true**&apikey=75fe1286